

Informe Tarea 1 – Grupo 14

Paula Vasquez, pvasquezj@usm.cl

Luciano Llancanao, lllancanao@usm.cl

Matilda Pinto Saud, mpintos@usm.cl

Emir Rojas, erojasg@usm.cl

Monserrat Rodríguez, mrodriguezag@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

La función lecturaSensor() simula la entrada de medición de un sensor. Esta función entrega datos enteros con valores entre 0 a 255. Inicialmente se almacenan este rango de valores en una variable interger que posee un tamaño de 4 bytes, pero la función entrega una variable double, esto permite una mayor precisión decimal.

La implementación de enum se debe a la necesidad de tener un código ordenado, comprensible y modular.

Los tres tipos de estado INIT, READ y SHOW, describen la misma función que cumplen, facilitando la comprensión del código. En vez de tener 3 funciones diferentes con la tarea de inicializar el sensor, leer su estado e imprimir su estado, se utiliza enum para que en un solo ámbito se gestione la lógica de sensores.

En la función lecturaSensor se declara lecturaAleatoria como una variable de tipo double y se le asigna un valor aleatorio entre 0 a 255. Finalmente, la función entrega la variable y termina su ejecución.

Figura N°1: Anexo A [pagina 6]

1.2. Diseño completo de la nueva función main()

La nueva función main() está representada como un diagrama de flujo por la figura 2:

En primer lugar, se inicializa una semilla que permite generar los números aleatorios que almacenaran los elementos del arreglo sensores[].

En segundo lugar, se reserva espacio en memoria para el arreglo con la cantidad de elementos fijada por NUM_SENSORES.

Por último, se hacen 3 llamados a la función usarSensores que tienen como parámetros el arreglo de sensores y en cada uno un tipo de tarea a realizar en el siguiente orden:

INIT: Se inicializan los sensores, a todo elemento del arreglo se le asigna el valor 0.

READ: Se realiza la lectura de los sensores, en donde se le asigna el valor simulado del sensor que fue obtenido mediante la función lecturaSensor() asignando los valores de salida a cada elemento del arreglo.

SHOW: Se muestra en pantalla el siguiente mensaje “Sensor (numero de sensor) (valor de la variable) [mV]” para cada sensor.

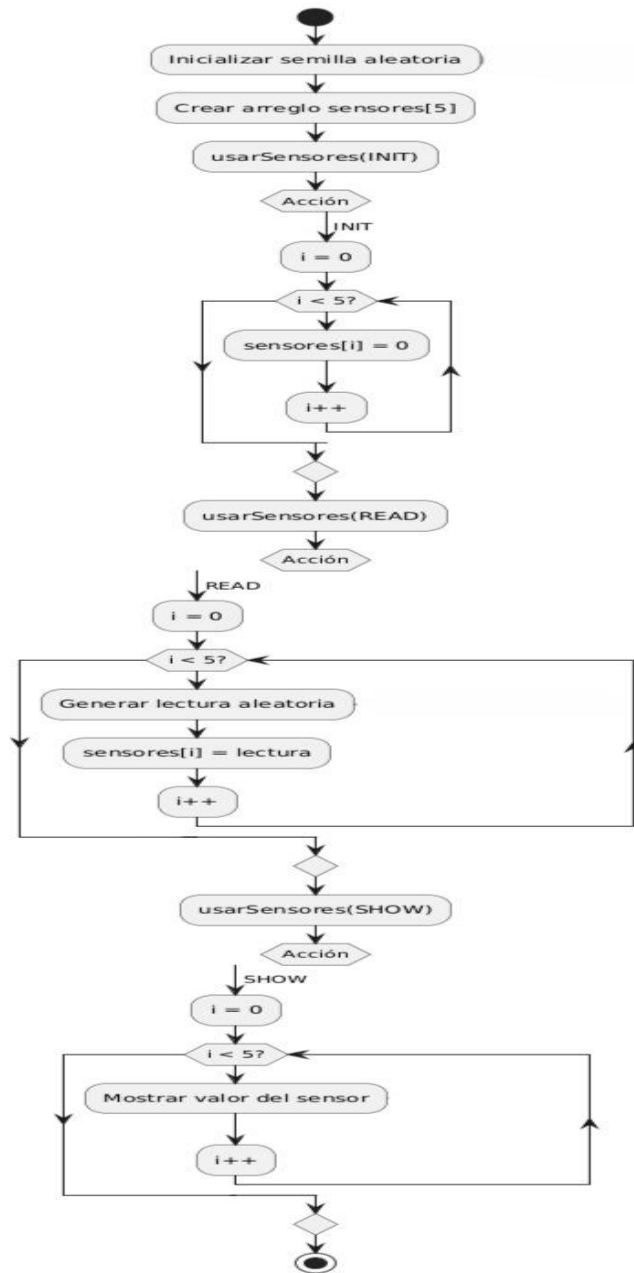


Figura N°2: Diagrama de flujo función main() .

2. Parte 2

2.1. Hito 1

1. Que es lo que retorna la función clock.

La función `clock()` retorna la cantidad de ticks de CPU que han transcurrido desde comenzó la ejecución del programa hasta el termino de éste.

2. Qué unidades tienen las variables inicio y fin.

Las variables inicio y fin almacenan los datos en forma de ticks de un reloj estos equivalen a 100 nanosegundos.

3. Por qué para obtener el tiempo total de duración del ciclo for se debe dividir la diferencia entre fin e inicio con el valor constante `CLOCKS_PER_SEC`.

Porque los datos almacenados están en ticks, entonces hay que convertirlos a segundos por medio de la división con la variable `CLOCKS_PER_SEC`, donde se realiza la conversión $segundos = \frac{ticks}{ticks\ por\ segundo}$, luego se multiplica por $1e9$, para convertir los segundos a nanosegundos.

4. Explicar que hace `(double)` como prefijo de `(fin - inicio)`

El `double` como prefijo realiza una conversión de los datos `(fin - inicio)` a datos de tipo `double`, esto para que, al realizar la división con la variable `CLOCKS_PER_SEC` sea una división de números decimales y no enteros

5. Explicar para que sirve utilizar `(double)` como prefijo de `(fin - inicio)`

Sirve para obtener un resultado con decimales y mayor precisión. Entonces usar `double` permite calcular correctamente tiempos pequeños y obtener valores precisos en nanosegundos.

2.2. Hito 2

La función `cicloPrueba(casot_, caso)` retorna el tiempo que se demora por cada iteración al imprimir cierto carácter. Para que se pueda ejecutar, solo requiere una variable de entrada del tipo `enum` que indica el carácter a analizar.

La utilidad del `switch` recae en redefinir la variable carácter según el caso especificado en la entrada de la función, para que el código sea más eficiente.

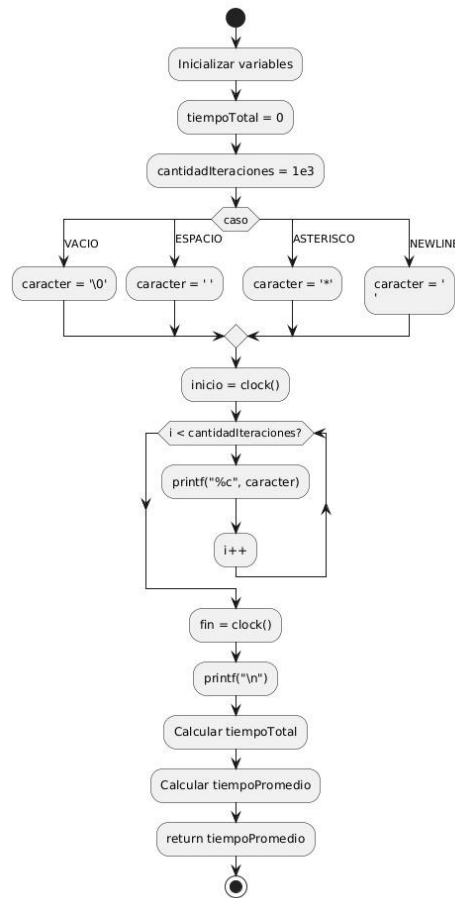


Figura N°3

Como podemos ver en la Figura N°3 primero inicializamos las variables necesarias para el cálculo del tiempo de iteración para luego definir los casos según la variable entrante de la función y después entrar al ciclo for() que correrá hasta alcanzar el numero especificado en la inicialización de variables.

Finalmente se realiza el cálculo del tiempo total para utilizarlo en el cálculo del tiempo promedio.

2.3. Hito 3

Ciclos	Vacio	Espacio	Asterisco	Newline
1,00E+02	20000	20000	50000	20000
5,00E+02	22000	20000	22000	24000
1,00E+03	20000	21000	21000	20000
5,00E+03	19800	21200	20400	23000
1,00E+04	20400	20400	20900	22900
5,00E+04	19540	20600	21120	21480
1,00E+05	26300	23670	24450	22660
5,00E+05	19368	20604	24574	21096
1,00E+06	19165	20684	21736	22037
5,00E+06	19699	20802	20832	21037
Promedio	20627,2	20896	24701,2	21821
DesvM	2143,59645	1047,56946	9010,12691	1330,89544

Tabla N °1

Para medir los tiempos de ejecución se usó como base la función cicloEjemplo() la cual crea otra función llamada cicloPrueba(caso_t caso).

Se variaron los números de ciclos de forma ascendente y se midieron los tiempos promedios (en nanosegundos) asociados a cada caso de carácter.

Podemos notar que el carácter con mayor desviación estándar fue el asterisco lo cual podría indicarnos un uso mayor de procesamiento para imprimir un carácter simple que por ejemplo simples espacios vacíos, espacios o saltos de línea.

Se realizaron 10 mediciones en un computador Lenovo con un procesador Ryzen 5 7520u, 8GB de memoria RAM y los gráficos integrados del procesador.

Se compilo el código en la ventana de CMD de Windows en la misma carpeta de creación.

3. Anexo A: explicando el diseño.

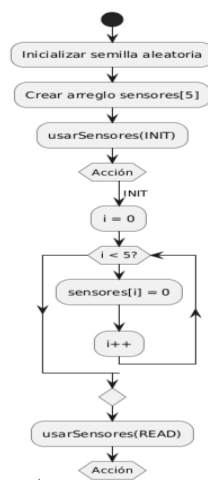


Figura N°1

En la Figura N°3 podemos apreciar el diagrama de flujo de la solución de la función `lecturaSensor()` la cual crea semillas que generan lecturas aleatorias, para después crear 5 sensores y ejecuta la multifunción `usarSensores()` que cambia su comportamiento según la acción que recibe la cual puede ser INIT, READ o SHOW.